TALLER DE EVALUACIÓN

Concepto y funcionamiento de una API
Unidad 3 · Consumo de APIs y manejo de datos externos

Asignatura	Programación Web Creativa II	**Docente**	Andrés Felipe Osorio Quintero
Unidad	3. Consumo de APIs y manejo de datos externos	**Subtema**	Concepto y funcionamiento de una API
Estudiante	Eliseth Velasco Mariana Diaz Samuel Ramirez Santiago Uran	**Fecha**	10 de noviembre de 2026

Parte I · Teoría (preguntas 1 a 5)

1 Explique con sus propias palabras por qué una API se describe como un «contrato» y no como una tecnología concreta. Apóyese en la analogía del restaurante e indique qué papel cumplen la carta, el mesero y la cocina.

Respuesta:

Una API se dice que es un contrato porque es un acuerdo entre dos partes: una pide las cosas de cierta forma y la otra se compromete a responder de cierta forma. No importa en qué lenguaje o con qué herramientas esté hecha por dentro. Lo que cuenta es que las dos partes cumplan lo acordado, y por eso no es una tecnología concreta.

En el restaurante, la carta es el contrato, porque ahí uno ve qué puede pedir y cómo pedirlo. El mesero es la API, que lleva el pedido a la cocina y trae la comida de vuelta. La cocina es el servidor, donde se prepara todo y que el cliente nunca ve. Si la cocina cambia la forma de cocinar pero la carta sigue igual, el cliente ni se da cuenta.

2 Argumente por qué el frontend moderno es, en buena medida, un consumidor de APIs. Desarrolle las tres razones prácticas por las que existen las APIs y relaciónelas con una aplicación web de su elección.

Respuesta:

Hoy en día casi ninguna página tiene todos sus datos guardados en su propio código. El frontend se encarga de mostrar las cosas y de recibir lo que hace el usuario, pero los datos están en servidores y los pide por medio de APIs. Por eso el frontend moderno es casi siempre un consumidor de APIs.

Como ejemplo escogí Spotify. Las tres razones por las que existen las APIs son estas:

1. Los datos están en otro lugar. Las canciones y las listas de reproducción están en los servidores de Spotify, y la app las trae con la API.

2. Separar las tareas. El frontend se preocupa porque la app se vea bien y sea fácil de usar, y el servidor se encarga de la música, las cuentas y los pagos. Así se puede cambiar una parte sin dañar la otra.

3. Reutilizar lo mismo en varios lugares. La misma API sirve para la página web, la app del celular y la del computador, y no hay que hacer todo de nuevo para cada una.

3 Describa el ciclo petición–respuesta entre cliente y servidor sobre HTTP: qué piezas componen una petición y una respuesta, y qué implica que HTTP sea un protocolo sin estado.

Respuesta:

El cliente, que casi siempre es el navegador, manda una petición. El servidor la recibe, la procesa y devuelve una respuesta. Eso es el ciclo petición–respuesta.

Una petición se compone del método (GET, POST, PUT, DELETE), la URL del recurso que se quiere, las cabeceras con información extra (como el tipo de contenido) y, a veces, un cuerpo con datos, por ejemplo cuando se envía un formulario.

Una respuesta trae el código de estado (200, 404, 500...), las cabeceras (como Content-Type) y el cuerpo con lo que se pidió, que muchas veces viene en formato JSON.

Que HTTP no tenga estado quiere decir que el servidor no se acuerda de las peticiones anteriores, y cada una la trata como si fuera la primera. Por eso, si hay que recordar algo, como quién inició sesión, el cliente tiene que mandar esa información en cada petición, por ejemplo con un token o una cookie.

4 Analice cómo el método HTTP y el código de estado orientan las decisiones del cliente. Explique qué es la idempotencia, por qué importa al decidir si se puede reintentar una petición y qué reacción corresponde a cada familia de códigos (2xx, 3xx, 4xx y 5xx).

Respuesta:

El método le dice al servidor qué queremos hacer: GET para leer, POST para crear, PUT para reemplazar y DELETE para borrar. El código de estado nos dice qué pasó. Con las dos cosas el cliente decide qué sigue: mostrar los datos, avisar de un error o intentar otra vez.

La idempotencia es cuando una petición se puede repetir varias veces y el resultado en el servidor es el mismo que si se hubiera hecho una sola vez. GET, PUT y DELETE son idempotentes, y POST no. Esto importa porque si una petición falla por la red y no sabemos si llegó, solo es seguro repetirla si es idempotente. Si repetimos un POST podríamos crear lo mismo dos veces, por ejemplo un pedido duplicado.

Según la familia del código, el cliente reacciona así:

• 2xx: todo salió bien, se muestran los datos o se confirma la acción.

• 3xx: el recurso cambió de lugar, entonces hay que ir a la nueva dirección que indica el servidor.

• 4xx: el error es de la petición (ruta mal escrita, falta de permisos, datos inválidos). No sirve repetirla igual, hay que corregirla o avisarle al usuario.

• 5xx: el error es del servidor. Se puede intentar más tarde, si el método es idempotente, o mostrar un mensaje de que hay un problema temporal.

5 Una interfaz que depende de la red puede fallar de varias formas. Explique por qué debe modelar de forma explícita sus estados (cargando, éxito, vacío y error), cómo se agrupan los errores y por qué fetch no rechaza la promesa ante un 404 o un 500.

Respuesta:

La red puede ser lenta o fallar, y los datos no siempre llegan al instante. Si no se piensan los estados, el usuario puede quedarse viendo una pantalla en blanco sin saber si está cargando, si no hay nada que mostrar o si algo salió mal. Por eso se manejan cuatro estados: cargando (se espera la respuesta), éxito (llegaron los datos), vacío (todo salió bien pero no hay datos) y error (algo falló). Cada uno debe tener su propio mensaje.

Los errores se pueden agrupar en tres tipos: los de red (no hay conexión o no se encuentra el servidor), los de HTTP (el servidor responde con un 4xx o un 5xx) y los de tiempo de espera (la respuesta se demora tanto que se cancela).

fetch solo rechaza la promesa cuando la petición no se pudo completar, por ejemplo si no hay internet. Un 404 o un 500 sí son respuestas del servidor, o sea que la comunicación funcionó, aunque el resultado fue un error. Por eso nosotros tenemos que revisar response.ok o response.status y lanzar el error por nuestra cuenta.

Parte II · Práctica (ejercicios 6 a 20)

Cada ejercicio está hecho en un archivo .html con el JavaScript dentro de la etiqueta <script>. Abajo van el código y el cuadro para la captura del resultado.

6 Primera petición con fetch

Enunciado. Una página necesita mostrar los datos de un usuario que vive en un servicio externo y no en su propio código.

Actividad. Con async/await, pida el usuario 1 a JSONPlaceholder (/users/1) y muestre en la página su nombre y su correo usando textContent.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 6</title></head>
<body>
  <h1>Datos del usuario</h1>
  <p id="nombre"></p>
  <p id="correo"></p>

  <script>
    async function cargarUsuario() {
      const respuesta = await fetch("https://jsonplaceholder.typicode.com/users/1");
      const usuario = await respuesta.json();
      document.getElementById("nombre").textContent = "Nombre: " + usuario.name;
      document.getElementById("correo").textContent = "Correo: " + usuario.email;
    }
    cargarUsuario();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

7 Inspeccionar la respuesta

Enunciado. Antes de confiar en unos datos conviene revisar cómo respondió el servidor.

Actividad. Pida el artículo 1 (/posts/1) y muestre en pantalla el código de estado, el valor de response.ok, la cabecera Content-Type y el título del artículo.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 7</title></head>
<body>
  <h1>Inspeccionar la respuesta</h1>
  <p id="estado"></p>
  <p id="ok"></p>
  <p id="tipo"></p>
  <p id="titulo"></p>

  <script>
    async function inspeccionar() {
      const respuesta = await fetch("https://jsonplaceholder.typicode.com/posts/1");
      const articulo = await respuesta.json();
```

```
      document.getElementById("estado").textContent = "Código de estado: " + respuesta.status;
      document.getElementById("ok").textContent = "response.ok: " + respuesta.ok;
      document.getElementById("tipo").textContent =
        "Content-Type: " + respuesta.headers.get("Content-Type");
      document.getElementById("titulo").textContent = "Título: " + articulo.title;
    }
    inspeccionar();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

8 Detectar un error HTTP

Enunciado. fetch no rechaza la promesa ante un 404 o un 500, pero la aplicación debe enterarse de que algo falló.

Actividad. Solicite un recurso que no exista (por ejemplo, /posts/99999), revise response.ok y muestre un mensaje con el código de estado recibido y la familia a la que pertenece.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 8</title></head>
<body>
  <h1>Detectar un error HTTP</h1>
  <p id="mensaje"></p>

  <script>
    function familiaDelEstado(codigo) {
      if (codigo >= 200 && codigo < 300) return "2xx (éxito)";
      if (codigo >= 300 && codigo < 400) return "3xx (redirección)";
      if (codigo >= 400 && codigo < 500) return "4xx (error del cliente)";
      if (codigo >= 500) return "5xx (error del servidor)";
      return "desconocida";
    }

    async function pedirRecursoInexistente() {
      const mensaje = document.getElementById("mensaje");
      const respuesta = await fetch("https://jsonplaceholder.typicode.com/posts/99999");

      if (!respuesta.ok) {
        mensaje.textContent = "Falló la petición. Código recibido: " + respuesta.status +
          " - Familia: " + familiaDelEstado(respuesta.status);
        return;
      }
      mensaje.textContent = "La petición salió bien.";
    }
    pedirRecursoInexistente();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

9 Parámetros de consulta

Enunciado. Muchas APIs filtran sus resultados mediante parámetros en la URL, como /posts?userId=1.

Actividad. Construya la URL con URLSearchParams para pedir las publicaciones de un autor que usted elija. Muestre en pantalla la URL final y cuántas publicaciones llegaron.

Nota: Autor elegido: usuario 3. La API devuelve 10 publicaciones por autor.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 9</title></head>
<body>
  <h1>Parámetros de consulta</h1>
  <p id="url"></p>
  <p id="cantidad"></p>

  <script>
    async function pedirPublicaciones(idAutor) {
      const parametros = new URLSearchParams({ userId: idAutor });
      const url = "https://jsonplaceholder.typicode.com/posts?" + parametros.toString();

      const respuesta = await fetch(url);
      const publicaciones = await respuesta.json();

      document.getElementById("url").textContent = "URL final: " + url;
      document.getElementById("cantidad").textContent =
        "Publicaciones recibidas: " + publicaciones.length;
    }
    pedirPublicaciones(3); // autor elegido: el usuario 3
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

10 Transformación de datos

Enunciado. Los datos externos rara vez tienen la forma que necesita la interfaz.

Actividad. Con las publicaciones de un autor, use map para conservar solo los títulos y filter para quedarse con los que cumplan un criterio propio (por ejemplo, una longitud mínima). Dibuje la lista resultante con textContent.

Nota: Criterio propio: se conservan los títulos con 30 caracteres o más.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 10</title></head>
<body>
  <h1>Títulos largos del autor 3</h1>
  <p id="resumen"></p>
  <ul id="lista"></ul>

  <script>
    const LONGITUD_MINIMA = 30; // criterio propio: títulos de 30 letras o más

    async function mostrarTitulos() {
      const url = "https://jsonplaceholder.typicode.com/posts?userId=3";
      const respuesta = await fetch(url);
      const publicaciones = await respuesta.json();

      const titulos = publicaciones.map(function (p) { return p.title; });
      const largos = titulos.filter(function (t) { return t.length >= LONGITUD_MINIMA; });

      document.getElementById("resumen").textContent =
        "De " + titulos.length + " títulos, " + largos.length +
        " tienen " + LONGITUD_MINIMA + " o más caracteres.";

      const lista = document.getElementById("lista");
      largos.forEach(function (titulo) {
        const item = document.createElement("li");
        item.textContent = titulo;
        lista.appendChild(item);
      });
    }
    mostrarTitulos();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

11 Selector dinámico de autores

Enunciado. Las opciones de una interfaz también pueden provenir de la API.

Actividad. Llene un <select> con los autores de /users y haga que, al cambiar la selección, la lista de publicaciones se actualice con los títulos del autor elegido.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 11</title></head>
<body>
  <h1>Selector dinámico de autores</h1>
  <label for="autores">Autor: </label>
  <select id="autores"></select>
  <ul id="lista"></ul>

  <script>
    const BASE = "https://jsonplaceholder.typicode.com";
    const selector = document.getElementById("autores");
    const lista = document.getElementById("lista");

    async function cargarAutores() {
      const respuesta = await fetch(BASE + "/users");
      const autores = await respuesta.json();
      autores.forEach(function (autor) {
        const opcion = document.createElement("option");
        opcion.value = autor.id;
        opcion.textContent = autor.name;
        selector.appendChild(opcion);
      });
      cargarPublicaciones(selector.value); // muestra las del primer autor
    }

    async function cargarPublicaciones(idAutor) {
      const respuesta = await fetch(BASE + "/posts?userId=" + idAutor);
      const publicaciones = await respuesta.json();
      lista.textContent = "";
      publicaciones.forEach(function (p) {
        const item = document.createElement("li");
        item.textContent = p.title;
        lista.appendChild(item);
      });
    }

    selector.addEventListener("change", function () {
      cargarPublicaciones(selector.value);
    });

    cargarAutores();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

12 Estados de la interfaz

Enunciado. Una interfaz que depende de la red atraviesa cuatro estados: cargando, éxito, vacío y error.

Actividad. Amplíe el ejercicio 11 para mostrar un mensaje distinto en cada estado y distinga el error con una clase CSS. Provoque el estado vacío con un autor sin publicaciones y el error desactivando la red en las herramientas de desarrollo.

Nota: Para el estado vacío se agregó una opción extra con el autor 11, que no tiene publicaciones. Para el estado de error se activa «Offline» en la pestaña Network de las herramientas de desarrollo.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8"><title>Ejercicio 12</title>
  <style>
    .error { color: #b00020; font-weight: bold; }
    .vacio { color: #666; font-style: italic; }
  </style>
</head>
<body>
  <h1>Estados de la interfaz</h1>
  <select id="autores"></select>
  <p id="estado"></p>
  <ul id="lista"></ul>

  <script>
    const BASE = "https://jsonplaceholder.typicode.com";
    const selector = document.getElementById("autores");
    const estado = document.getElementById("estado");
    const lista = document.getElementById("lista");

    function mostrarEstado(texto, clase) {
      estado.textContent = texto;
      estado.className = clase || "";
    }

    async function cargarAutores() {
      try {
        const respuesta = await fetch(BASE + "/users");
        if (!respuesta.ok) throw new Error("Error HTTP " + respuesta.status);
        const autores = await respuesta.json();
        autores.forEach(function (autor) {
          const opcion = document.createElement("option");
          opcion.value = autor.id;
          opcion.textContent = autor.name;
          selector.appendChild(opcion);
        });
        // autor 11 no existe en la API: sirve para provocar el estado vacío
        const sinPosts = document.createElement("option");
        sinPosts.value = 11;
        sinPosts.textContent = "Autor sin publicaciones";
        selector.appendChild(sinPosts);
        cargarPublicaciones(selector.value);
      } catch (error) {
        mostrarEstado("Error: no se pudieron cargar los autores.", "error");
      }
    }

    async function cargarPublicaciones(idAutor) {
      lista.textContent = "";
      mostrarEstado("Cargando publicaciones...");              // estado: cargando
      try {
        const respuesta = await fetch(BASE + "/posts?userId=" + idAutor);
        if (!respuesta.ok) throw new Error("Error HTTP " + respuesta.status);
        const publicaciones = await respuesta.json();

        if (publicaciones.length === 0) {                      // estado: vacío
          mostrarEstado("Este autor no tiene publicaciones.", "vacio");
          return;
        }
        publicaciones.forEach(function (p) {                   // estado: éxito
          const item = document.createElement("li");
          item.textContent = p.title;
          lista.appendChild(item);
        });
        mostrarEstado("Se cargaron " + publicaciones.length + " publicaciones.");
      } catch (error) {                                        // estado: error
        mostrarEstado("Error: revise su conexión e intente de nuevo.", "error");
```

```
      }
    }

    selector.addEventListener("change", function () {
      cargarPublicaciones(selector.value);
    });
    cargarAutores();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

13 Familias de error

Enunciado. Los errores de una petición no son todos iguales: pueden ser de red, de HTTP o de tiempo de espera.

Actividad. Cree una clase de error propia para los estados 4xx y 5xx y una función que reciba cualquier error y devuelva un mensaje comprensible según su familia. Provoque cada caso y muestre el mensaje en la página.

Nota: Cada botón provoca una familia: 404 (4xx), 500 (5xx, usando httpbin.org), dominio inexistente (red) y límite de 1 ms (tiempo agotado).

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8"><title>Ejercicio 13</title>
  <style> .error { color: #b00020; font-weight: bold; } </style>
</head>
<body>
  <h1>Familias de error</h1>
  <button data-caso="404">Provocar 4xx</button>
  <button data-caso="500">Provocar 5xx</button>
  <button data-caso="red">Provocar error de red</button>
  <button data-caso="tiempo">Provocar tiempo agotado</button>
  <p id="mensaje" class="error"></p>

  <script>
    // Clase de error propia para respuestas 4xx y 5xx
    class ErrorHTTP extends Error {
      constructor(estado) {
        super("Error HTTP " + estado);
        this.name = "ErrorHTTP";
        this.estado = estado;
      }
    }

    // Recibe cualquier error y devuelve un mensaje fácil de entender
    function mensajeDeError(error) {
      if (error instanceof ErrorHTTP) {
        if (error.estado >= 500) return "El servidor tuvo un problema (" + error.estado + "). Intente más tarde.";
        return "La petición no es válida o el recurso no existe (" + error.estado + ").";
      }
      if (error.name === "AbortError") return "La petición tardó demasiado y fue cancelada.";
      if (error instanceof TypeError) return "Error de red: no hay conexión con el servidor.";
      return "Ocurrió un error inesperado.";
    }

    async function pedir(url, limiteMs) {
      const controlador = new AbortController();
```

```
      const temporizador = setTimeout(function () { controlador.abort(); }, limiteMs);
      try {
        const respuesta = await fetch(url, { signal: controlador.signal });
        if (!respuesta.ok) throw new ErrorHTTP(respuesta.status);
        return await respuesta.json();
      } finally {
        clearTimeout(temporizador);
      }
    }

    const casos = {
      "404": { url: "https://jsonplaceholder.typicode.com/posts/99999", limite: 8000 },
      "500": { url: "https://httpbin.org/status/500", limite: 8000 },
      "red": { url: "https://dominio-que-no-existe-12345.invalid/datos", limite: 8000 },
      "tiempo": { url: "https://jsonplaceholder.typicode.com/posts", limite: 1 }
    };

    document.addEventListener("click", async function (evento) {
      const caso = evento.target.dataset.caso;
      if (!caso) return;
      const mensaje = document.getElementById("mensaje");
      mensaje.textContent = "Probando...";
      try {
        await pedir(casos[caso].url, casos[caso].limite);
        mensaje.textContent = "No hubo error.";
      } catch (error) {
        mensaje.textContent = mensajeDeError(error);
      }
    });
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

14 Tiempo límite con AbortController

Enunciado. fetch no tiene tiempo límite propio: una conexión colgada espera indefinidamente.

Actividad. Cancele la petición con AbortController si supera un límite en milisegundos definido por usted y cancele el temporizador con finally. Pruebe con un límite tan corto que se agote y muestre el mensaje correspondiente.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 14</title></head>
<body>
  <h1>Tiempo límite con AbortController</h1>
  <p id="mensaje"></p>

  <script>
    const LIMITE_MS = 1; // límite tan corto que siempre se agota (para la prueba)

    async function pedirConLimite() {
      const mensaje = document.getElementById("mensaje");
      const controlador = new AbortController();
      const temporizador = setTimeout(function () {
        controlador.abort();
      }, LIMITE_MS);

      try {
        const respuesta = await fetch("https://jsonplaceholder.typicode.com/posts", {
          signal: controlador.signal
```

```
        });
        const datos = await respuesta.json();
        mensaje.textContent = "Llegaron " + datos.length + " publicaciones a tiempo.";
      } catch (error) {
        if (error.name === "AbortError") {
          mensaje.textContent = "Tiempo agotado: la petición superó los " + LIMITE_MS + " ms y se canceló.";
        } else {
          mensaje.textContent = "Ocurrió otro error: " + error.message;
        }
      } finally {
        clearTimeout(temporizador); // se limpia el temporizador pase lo que pase
      }
    }
    pedirConLimite();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

15 Centralizar el acceso a la API

Enunciado. Una buena práctica es resolver en un solo lugar la URL base, la revisión de response.ok y el tiempo límite.

Actividad. Cree la función pedirJSON(ruta, parametros, limiteMs) que reúna esas tareas y refactorice el ejercicio 12 para que todas sus peticiones pasen por ella.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8"><title>Ejercicio 15</title>
  <style>
    .error { color: #b00020; font-weight: bold; }
    .vacio { color: #666; font-style: italic; }
  </style>
</head>
<body>
  <h1>Acceso centralizado a la API</h1>
  <select id="autores"></select>
  <p id="estado"></p>
  <ul id="lista"></ul>

  <script>
    const URL_BASE = "https://jsonplaceholder.typicode.com";

    // Un solo lugar para: URL base, revisión de response.ok y tiempo límite
    async function pedirJSON(ruta, parametros = {}, limiteMs = 8000) {
      const texto = new URLSearchParams(parametros).toString();
      const url = URL_BASE + ruta + (texto ? "?" + texto : "");

      const controlador = new AbortController();
      const temporizador = setTimeout(function () { controlador.abort(); }, limiteMs);
      try {
        const respuesta = await fetch(url, { signal: controlador.signal });
        if (!respuesta.ok) throw new Error("Error HTTP " + respuesta.status);
        return await respuesta.json();
      } finally {
        clearTimeout(temporizador);
      }
    }

    const selector = document.getElementById("autores");
```

```
    const estado = document.getElementById("estado");
    const lista = document.getElementById("lista");

    function mostrarEstado(texto, clase) {
      estado.textContent = texto;
      estado.className = clase || "";
    }

    async function cargarAutores() {
      try {
        const autores = await pedirJSON("/users");
        autores.forEach(function (autor) {
          const opcion = document.createElement("option");
          opcion.value = autor.id;
          opcion.textContent = autor.name;
          selector.appendChild(opcion);
        });
        const sinPosts = document.createElement("option");
        sinPosts.value = 11;
        sinPosts.textContent = "Autor sin publicaciones";
        selector.appendChild(sinPosts);
        cargarPublicaciones(selector.value);
      } catch (error) {
        mostrarEstado("Error: no se pudieron cargar los autores.", "error");
      }
    }

    async function cargarPublicaciones(idAutor) {
      lista.textContent = "";
      mostrarEstado("Cargando publicaciones...");
      try {
        const publicaciones = await pedirJSON("/posts", { userId: idAutor }, 5000);
        if (publicaciones.length === 0) {
          mostrarEstado("Este autor no tiene publicaciones.", "vacio");
          return;
        }
        publicaciones.forEach(function (p) {
          const item = document.createElement("li");
          item.textContent = p.title;
          lista.appendChild(item);
        });
        mostrarEstado("Se cargaron " + publicaciones.length + " publicaciones.");
      } catch (error) {
        mostrarEstado("Error: revise su conexión e intente de nuevo.", "error");
      }
    }

    selector.addEventListener("change", function () {
      cargarPublicaciones(selector.value);
    });
    cargarAutores();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

16 Crear un recurso con POST

Enunciado. Para enviar datos hay que declarar su formato y serializar el objeto.

Actividad. Envíe un POST a /posts con un objeto creado por usted, usando la cabecera Content-Type y JSON.stringify. Muestre el estado y la respuesta, y compruebe si el recurso persiste al recargar la página.

Nota: JSONPlaceholder simula la creación y devuelve el id 101, pero no guarda nada de verdad. Al recargar o pedir /posts/101, responde 404, así que el recurso no persiste.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 16</title></head>
<body>
  <h1>Crear un recurso con POST</h1>
  <pre id="salida"></pre>

  <script>
    const BASE = "https://jsonplaceholder.typicode.com";

    async function crearPublicacion() {
      const salida = document.getElementById("salida");
      const nuevaPublicacion = {
        title: "Mi primera publicación desde fetch",
        body: "Estoy practicando el método POST con JSONPlaceholder.",
        userId: 1
      };

      const respuesta = await fetch(BASE + "/posts", {
        method: "POST",
        headers: { "Content-Type": "application/json; charset=UTF-8" },
        body: JSON.stringify(nuevaPublicacion)
      });
      const creada = await respuesta.json();

      let texto = "Método: POST\nEstado: " + respuesta.status +
        "\nRespuesta:\n" + JSON.stringify(creada, null, 2);

      // Comprobar si el recurso quedó guardado: pedimos el id que nos devolvió
      const verificacion = await fetch(BASE + "/posts/" + creada.id);
      texto += "\n\nGET /posts/" + creada.id + " -> estado " + verificacion.status +
        (verificacion.ok ? " (el recurso existe)" : " (el recurso NO persiste)");

      salida.textContent = texto;
    }
    crearPublicacion();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

17 Actualizar y eliminar

Enunciado. Los métodos PUT y DELETE operan sobre un recurso concreto y son idempotentes.

Actividad. Envíe un PUT con cuerpo JSON y un DELETE a /posts/1. Muestre para cada uno el método, el estado y la respuesta, e indique en pantalla si el método es idempotente.

Nota: PUT y DELETE son idempotentes: repetirlos deja el recurso en el mismo estado. En JSONPlaceholder tampoco se modifica nada de verdad.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 17</title></head>
<body>
  <h1>Actualizar y eliminar</h1>
  <pre id="salida"></pre>

  <script>
    const BASE = "https://jsonplaceholder.typicode.com";
    const salida = document.getElementById("salida");

    async function actualizarYEliminar() {
      // PUT: reemplaza el recurso completo
      const resPut = await fetch(BASE + "/posts/1", {
        method: "PUT",
        headers: { "Content-Type": "application/json; charset=UTF-8" },
        body: JSON.stringify({
          id: 1,
          title: "Título actualizado",
          body: "Contenido actualizado con PUT.",
          userId: 1
        })
      });
      const datosPut = await resPut.json();
      salida.textContent += "Método: PUT\nEstado: " + resPut.status +
        "\nRespuesta: " + JSON.stringify(datosPut, null, 2) +
        "\n¿Es idempotente? Sí\n\n";

      // DELETE: elimina el recurso
      const resDelete = await fetch(BASE + "/posts/1", { method: "DELETE" });
      const datosDelete = await resDelete.json();
      salida.textContent += "Método: DELETE\nEstado: " + resDelete.status +
        "\nRespuesta: " + JSON.stringify(datosDelete, null, 2) +
        "\n¿Es idempotente? Sí";
    }
    actualizarYEliminar();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

18 Cliente REST de práctica

Enunciado. Un único manejador de eventos puede atender varios botones si cada uno declara su método.

Actividad. Construya una página con un botón por método (GET, POST, PUT y DELETE) usando atributos data-metodo y un solo manejador de clics. Muestre en un <pre> el método, el estado y la respuesta con JSON.stringify(datos, null, 2), y capture los errores de red.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 18</title></head>
<body>
  <h1>Cliente REST de práctica</h1>
  <button data-metodo="GET">GET</button>
  <button data-metodo="POST">POST</button>
  <button data-metodo="PUT">PUT</button>
  <button data-metodo="DELETE">DELETE</button>
  <pre id="salida">Pulse un botón para probar un método.</pre>

  <script>
    const URL_POST = "https://jsonplaceholder.typicode.com/posts";
    const cuerpo = JSON.stringify({ id: 1, title: "Prueba", body: "Texto de prueba", userId: 1 });

    // Un solo manejador para los cuatro botones
    document.addEventListener("click", async function (evento) {
      const metodo = evento.target.dataset.metodo;
      if (!metodo) return; // se ignora si no fue un botón de método

      const salida = document.getElementById("salida");
      salida.textContent = "Enviando " + metodo + "...";

      const opciones = { method: metodo };
      let url = URL_POST;
      if (metodo !== "POST") url = URL_POST + "/1"; // GET, PUT y DELETE usan un recurso
      if (metodo === "POST" || metodo === "PUT") {
        opciones.headers = { "Content-Type": "application/json; charset=UTF-8" };
        opciones.body = cuerpo;
      }

      try {
        const respuesta = await fetch(url, opciones);
        const datos = await respuesta.json();
        salida.textContent = "Método: " + metodo + "\nEstado: " + respuesta.status +
          "\nRespuesta:\n" + JSON.stringify(datos, null, 2);
      } catch (error) {
        salida.textContent = "Error de red: no se pudo completar la petición " + metodo + ".";
      }
    });
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

19 Clima actual con Open-Meteo

Enunciado. Open-Meteo es una API pública que no exige clave en su uso no comercial.

Actividad. Construya con URLSearchParams la petición para una ciudad colombiana de su elección (latitud, longitud, variables actuales y zona horaria). Muestre la temperatura, la humedad y el viento con sus unidades.

Nota: Ciudad elegida: Manizales (latitud 5.07, longitud -75.52).

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Ejercicio 19</title></head>
<body>
  <h1>Clima actual en Manizales</h1>
  <p id="temperatura"></p>
  <p id="humedad"></p>
  <p id="viento"></p>

  <script>
    async function cargarClima() {
      const parametros = new URLSearchParams({
        latitude: 5.07,
        longitude: -75.52,
        current: "temperature_2m,relative_humidity_2m,wind_speed_10m",
        timezone: "America/Bogota"
      });
      const url = "https://api.open-meteo.com/v1/forecast?" + parametros.toString();

      const respuesta = await fetch(url);
      if (!respuesta.ok) {
        document.getElementById("temperatura").textContent = "Error HTTP " + respuesta.status;
        return;
      }
      const datos = await respuesta.json();
      const actual = datos.current;
      const unidades = datos.current_units;

      document.getElementById("temperatura").textContent =
        "Temperatura: " + actual.temperature_2m + " " + unidades.temperature_2m;
      document.getElementById("humedad").textContent =
        "Humedad: " + actual.relative_humidity_2m + " " + unidades.relative_humidity_2m;
      document.getElementById("viento").textContent =
        "Viento: " + actual.wind_speed_10m + " " + unidades.wind_speed_10m;
    }
    cargarClima();
  </script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

20 Panel del clima

Enunciado. El caso integrador reúne consumo, parámetros, transformación, manejo de errores y renderizado seguro.

Actividad. Amplíe el ejercicio 19 con un selector de varias ciudades y el pronóstico de tres días: convierta los arreglos paralelos en una lista de objetos y dibuje una tarjeta por día con textContent, con estados de carga y error. Añada comentarios que señalen qué función es la extracción, cuál la transformación y cuál la «carga» efímera.

Evidencia 1 · Código desarrollado en Visual Studio Code

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8"><title>Ejercicio 20 - Panel del clima</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    #tarjetas { display: flex; gap: 12px; flex-wrap: wrap; margin-top: 12px; }
    .tarjeta { border: 1px solid #999; border-radius: 8px; padding: 10px; width: 170px; }
    .error { color: #b00020; font-weight: bold; }
  </style>
</head>
<body>
  <h1>Panel del clima</h1>
  <select id="ciudad"></select>
  <p id="estado"></p>
  <p id="actual"></p>
  <div id="tarjetas"></div>

  <script>
    const CIUDADES = {
      "Manizales": { lat: 5.07, lon: -75.52 },
      "Bogotá": { lat: 4.71, lon: -74.07 },
      "Medellín": { lat: 6.24, lon: -75.58 },
      "Cali": { lat: 3.45, lon: -76.53 },
      "Cartagena": { lat: 10.39, lon: -75.48 }
    };

    // ===== EXTRACCIÓN: pide los datos crudos a la API =====
    async function extraerClima(ciudad) {
      const parametros = new URLSearchParams({
        latitude: ciudad.lat,
        longitude: ciudad.lon,
        current: "temperature_2m,relative_humidity_2m,wind_speed_10m",
        daily: "temperature_2m_max,temperature_2m_min,precipitation_sum",
        timezone: "America/Bogota",
        forecast_days: 3
      });
      const controlador = new AbortController();
      const temporizador = setTimeout(function () { controlador.abort(); }, 8000);
      try {
        const respuesta = await fetch(
          "https://api.open-meteo.com/v1/forecast?" + parametros.toString(),
          { signal: controlador.signal }
        );
        if (!respuesta.ok) throw new Error("Error HTTP " + respuesta.status);
        return await respuesta.json();
      } finally {
        clearTimeout(temporizador);
      }
    }

    // ===== TRANSFORMACIÓN: convierte los arreglos paralelos en una lista de objetos =====
    function transformarPronostico(datos) {
      const d = datos.daily;
      return d.time.map(function (fecha, i) {
        return {
          fecha: fecha,
          maxima: d.temperature_2m_max[i],
          minima: d.temperature_2m_min[i],
          lluvia: d.precipitation_sum[i]
        };
      });
    }

    // ===== CARGA (efímera): dibuja en la página; se pierde al recargar =====
    function dibujarClima(datos, dias) {
      const u = datos.current_units;
      document.getElementById("actual").textContent =
        "Ahora: " + datos.current.temperature_2m + " " + u.temperature_2m +
        " | Humedad: " + datos.current.relative_humidity_2m + " " + u.relative_humidity_2m +
```

```
      " | Viento: " + datos.current.wind_speed_10m + " " + u.wind_speed_10m;

    const contenedor = document.getElementById("tarjetas");
    contenedor.textContent = "";
    dias.forEach(function (dia) {
      const tarjeta = document.createElement("div");
      tarjeta.className = "tarjeta";
      const titulo = document.createElement("strong");
      titulo.textContent = dia.fecha;
      const detalle = document.createElement("p");
      detalle.textContent = "Máx: " + dia.maxima + "°C - Mín: " + dia.minima +
        "°C - Lluvia: " + dia.lluvia + " mm";
      tarjeta.appendChild(titulo);
      tarjeta.appendChild(detalle);
      contenedor.appendChild(tarjeta);
    });
  }

  function mostrarEstado(texto, clase) {
    const estado = document.getElementById("estado");
    estado.textContent = texto;
    estado.className = clase || "";
  }

  async function actualizarPanel(nombre) {
    mostrarEstado("Cargando pronóstico de " + nombre + "...");   // estado: cargando
    document.getElementById("tarjetas").textContent = "";
    document.getElementById("actual").textContent = "";
    try {
      const datos = await extraerClima(CIUDADES[nombre]);
      const dias = transformarPronostico(datos);
      dibujarClima(datos, dias);
      mostrarEstado("Pronóstico de " + nombre + " listo.");
    } catch (error) {
      const texto = error.name === "AbortError"
        ? "La petición tardó demasiado."
        : "No se pudo obtener el clima. Revise su conexión.";
      mostrarEstado("Error: " + texto, "error");                  // estado: error
    }
  }

  const selector = document.getElementById("ciudad");
  Object.keys(CIUDADES).forEach(function (nombre) {
    const opcion = document.createElement("option");
    opcion.value = nombre;
    opcion.textContent = nombre;
    selector.appendChild(opcion);
  });
  selector.addEventListener("change", function () { actualizarPanel(selector.value); });
  actualizarPanel(selector.value);
</script>
</body>
</html>
```

Evidencia 2 · Resultado visualizado en el navegador

Pegue aquí la captura del resultado en el navegador

Enlace de GitHub (opcional): _____

ENTREGA FINAL · Enlace del repositorio de GitHub con todos los ejercicios

Pegue aquí el enlace: